

Exercise 2: Memory Matching Flutter Game

COMP5450 Mobile Programming

Student: Liang Mingchen

GitHub: <https://github.com/achenachena/comp5450-exercise-2-memory-match>

1. Assignment overview

This submission is a Flutter/Dart mobile app for Exercise 2. The handout asks for a memory matching game with custom styling, a README.pdf that explains how to configure and run the project, screenshots, and an exact description of the project structure.

2. Application description

The app title is **Memory Match**. The main screen shows a 4×5 grid (20 cards, 10 pairs). Each pair uses the same animal emoji. Card backs use bright colors (green, purple, blue, yellow, orange, pink) with a red question mark. When a card is tapped, it flips to show the animal.

- Subtitle text: “Find all matching pairs! ■”.
- Move counter in the top-right (increments when two cards are compared).
- Matched pair counter under the subtitle.
- Restart button at the bottom reshuffles the deck.
- Win dialog appears when all 10 pairs are cleared.
- No database or internet connection is required.

3. User interaction flow

- Player taps the first card → card flips face-up.
- Player taps a second card → move count increases by 1.
- If symbols match → both cards stay up and are marked matched.
- If symbols do not match → both flip back after ~750 ms.
- When all pairs are matched → win dialog with final move count.
- Restart → new shuffle, moves reset to 0.

4. Configure the development environment

Follow these steps on a new machine before running the project.

- Install the Flutter SDK (stable): <https://docs.flutter.dev/get-started/install>
- Install Android Studio and enable the Flutter + Dart plugins.
- On macOS for iOS builds: install Xcode and accept licenses if needed.
- Run flutter doctor in a terminal and resolve any failed checks.
- For Android: run flutter doctor --android-licenses if licenses are missing.
- Clone or copy the project, cd into exercise-2, run flutter pub get.
- Open the project root (folder with pubspec.yaml) in Android Studio via File → Open.
- Create or start an Android Virtual Device, or connect a USB phone with debugging on.

5. Run the solution

Option A — Android Studio

- Open the project and wait for sync to finish.
- Select a device from the toolbar dropdown.
- Click Run. The app installs and opens on the emulator or phone.

Option B — Command line

```
cd exercise-2
flutter pub get
flutter run
```

Build a debug APK

```
flutter build apk --debug
```

APK path: build/app/outputs/flutter-apk/app-debug.apk

Chrome (flutter run -d chrome) works for a quick UI check, but the course rubric notes a small penalty for demoing web only. Use an Android emulator or physical device for presentation and grading.

6. Verification commands

```
flutter analyze
flutter test
flutter devices
```

Expected result: analyze reports no issues; all unit and widget tests pass.

7. Exact project structure

```
exercise-2/  
  lib/main.dart  
  lib/models/memory_card_data.dart  
  lib/game/memory_game.dart  
  lib/screens/memory_match_screen.dart  
  lib/widgets/memory_card_tile.dart  
  test/memory_game_test.dart  
  test/widget_test.dart  
  android/          (Android Gradle project)  
  ios/              (Xcode project)  
  web/              (web entry)  
  linux/ macos/ windows/  
  pubspec.yaml  
  analysis_options.yaml  
  README.md  
  README.pdf  
  screenshot-home.png  
  screenshot-play.png
```

8. Source file responsibilities

File	Purpose
lib/main.dart	App entry; MaterialApp theme; loads MemoryMatchScreen
lib/models/memory_card_data.dart	Card model: id, symbol, colors, face/matched state
lib/game/memory_game.dart	Deck build, shuffle, tap handling, moves, win logic
lib/screens/memory_match_screen.dart	Scaffold, app bar, 4×5 grid, restart, dialogs
lib/widgets/memory_card_tile.dart	Card rendering and Y-axis flip animation
test/memory_game_test.dart	Tests pairs, moves, mismatch delay, reset
test/widget_test.dart	Tests screen load, restart, back-button snackbar

9. Game logic summary

- reset() builds 20 cards (two of each of 10 animals) and shuffles them.
- handleTap() ignores input while isBusy or after the game is won.
- First tap of a turn only flips a card; second tap triggers comparison.
- Matching cards set isMatched = true; when all cards match, isWon = true.
- Non-matching cards wait 750 ms, then flip face-down again.

10. Screenshots

The images below show the app running on a phone-sized viewport.

Figure 1 — Home screen (face-down cards)



Figure 2 — Gameplay (flipped cards)



Memory Match

1

Find all matching pairs! 🐾
Matched: 0 / 10

?	?	?	?
?	?	?	?
?	?	?	?
?	?	?	?
?	?	?	?

🔄 Restart